CS 590-Algorithms

Homework-4

Shree Shingre

Q.1

Given two arrays 'left' and 'right' of size n each. We build a MinHeap from left and a MaxHeap from right, then repeatedly while both heaps are non-empty and the minimum in left is less than the maximum in right: remove the max from the right heap and the min from the left heap and update 'sum'. Return sum.

Step-by-step analysis:

1. Building the heaps:

- If the implementation uses heapify, constructing a heap from an array of size n takes Θ(n) time. So MinHeap(left) is Θ(n) and MaxHeap(right) is Θ(n).

2. The while loop:

- Each iteration does two delete operations: rheap.DeleteMax() and lheap.DeleteMin(). Each heap delete takes O(log m) where m is the heap size; since sizes are ≤ n, each delete is O(log n).

- Each iteration removes one element from each heap, so the number of iterations is at most n (the heaps start with n elements and each iteration decreases both sizes by 1).

3. Total cost of the while loop:

- At most n iterations × O(log n) per iteration = O(n log n).

4. Add the heap-build cost:

- Heap construction cost $\Theta(n)$ + loop cost $O(n \log n)$ = $O(n \log n)$.

Conclusion: The worst-case time complexity of HeapMystery is $\Theta(n \log n)$.

2. Best-case time complexity of HeapMystery

Best-case scenario:

The while loop condition fails immediately, which happens if either lheap is empty or lheap.peek() ≥ rheap.peek() right after building the heaps. In that case we do no Delete operations.

Cost:

- Building both heaps (heapify) takes $\Theta(n)$ time.
- A constant number of heap peek checks and the while condition check are $\Theta(1)$.

Conclusion: Under the standard heapify assumption, the best-case time complexity is $\Theta(n)$.

3. Worst-case complexity of QueueMystery

Restated algorithm (with inner-loop index renamed for clarity):

```
q = Queue()
q.Enqueue(1)
for i = 1 to n do
    size = |q|
    for j = 1 to size do
        t = q.Dequeue()
        q.Enqueue(t)
        q.Enqueue(t + size)
    end
end
return q
```

Step-by-step analysis:

1. Growth of |q|:

- Initially |q| = 1.

- In one outer iteration, each of the current 'size' elements is dequeued and two elements are enqueued, so the queue size increases by exactly 'size'. That is, after processing 'size' elements the new size becomes size + size = 2×size.

- Thus the queue size doubles on each outer iteration. After i outer iterations the size is 2^i (with size before iteration i equal to 2^{i-1}).

2. Work done (number of inner iterations):

- On outer iteration i (1-based) the inner loop runs 2^{i-1} times.

- Total number of inner-loop iterations = Σ_{i=1..n} 2^{i-1} = 2^n − 1 = Θ(2^n). Each inner iteration does O(1) queue operations (a dequeue and two enqueues).

Conclusion: The worst-case time complexity is Θ(2^n). Note also the space used grows to Θ(2^n).

4. Add left and right children ranges for the BST leaves

The tree (values shown) is:

```
      25
     /  \
    15   35
   /  \ /  \
  10  20 30  40
```

Leaves: 10, 20, 30, 40. For each leaf we add two (empty) children that indicate the integer ranges that would be inserted at that position.

We use [min,max] (inclusive integer bounds) and −∞, ∞ for unbounded sides.

Ranges:

- For leaf 10:

  • left child range = [−∞, 9] (any integer strictly less than 10).

  • right child range = [11, 14] (greater than 10 but less than 15, because 15 is the ancestor that bounds this subtree).

- For leaf 20:

  • left child range = [16, 19] (greater than 15 and less than 20).

  • right child range = [21, 24] (greater than 20 and less than 25).

- For leaf 30:

  • left child range = [26, 29] (greater than 25 and less than 30).

  • right child range = [31, 34] (greater than 30 and less than 35).

- For leaf 40:

  • left child range = [36, 39] (greater than 35 and less than 40).

  • right child range = [41, ∞] (any integer strictly greater than 40).

Remark: The intervals are inclusive on integer endpoints and reflect the strict inequalities used by BST insertion (e.g., a value equal to 15 would be ignored because 15 already exists).

5. BST insert pseudocode that updates pred and succ

Idea (short): While descending the tree to find the insertion spot, keep track of the last node smaller than x (pred) and the last node larger than x (succ). After creating the new node, hook it into the pred–succ doubly-linked chain and attach it as the correct child of its parent.

Pseudocode (assume BST is non-empty and nodes have fields: data, left, right, pred, succ):

```
BSTInsertWithPredSucc(root, x)
    parent = nil
    cur = root
```

```
pred = nil        // last node seen with value < x
succ = nil        // last node seen with value > x

// search for insertion point while tracking pred and succ
while cur != nil do
    parent = cur
    if x < cur.data then
        succ = cur        // cur is a candidate successor
        cur = cur.left
    else if x > cur.data then
        pred = cur        // cur is a candidate predecessor
        cur = cur.right
    else
        return            // x already in tree: ignore insertion
end while

// create new node
new = NewNode(x)
new.left = nil
new.right = nil
new.pred = pred
new.succ = succ

// fix predecessor / successor links
if pred != nil then
    pred.succ = new
end if
if succ != nil then
    succ.pred = new
end if

// attach new node in the BST as child of parent
if parent == nil then
    // (not expected here because we assume non-empty tree)
    root = new
else if x < parent.data then
    parent.left = new
else
```

```
        parent.right = new
    end if

    return
```

Explanation:

- pred and succ track the nearest smaller and larger elements encountered while searching. These are exactly the predecessor and successor in sorted order for the new element when the tree has unique values.

- Updating pred.succ and succ.pred inserts the new node into the in-order doubly-linked chain in Θ(1) time.

- The overall insertion time is Θ(h) where h is the tree height (same as normal BST insert).

6. Θ(n) algorithm to return all "interesting" elements

Restatement: An element x in the array 'data' (unique integers) is interesting if the array contains a contiguous (by value) range of integers of length ≥ 2 that includes x and x is either the minimum or maximum value of that contiguous numeric range. Example: if data contains {3,4,5} then 3 and 5 are interesting.

Algorithm (LSD-style explanation):

1. Put all values into a hash set S for O(1) membership tests.

2. For each element v in data, check whether v − 1 is in S. If v − 1 not in S, then v is the start (minimum) of a numeric consecutive run. Walk forward while v + k is in S to find the end of that run. Let length = number of members in the run.

3. If length ≥ 2, add the run's endpoints (start v and end v+length−1) to the output list. (Make sure to avoid duplicates if two starts produce the same endpoint; you can use a set for the output.)

This is the classic O(n) algorithm for finding consecutive integer runs in a set.

Pseudocode:

```
FindInteresting(data)
    S = new HashSet()
    for x in data do
        S.add(x)
    end for

    result = new List()    // or HashSet to avoid duplicates

    for x in data do
        // only start walks at the beginning of a run
        if (x - 1) not in S then
            // x is the start of a consecutive run
            length = 1
            while (x + length) in S do
                length = length + 1
            end while
            if length >= 2 then
                result.add(x)                // smallest in run
                result.add(x + length - 1)    // largest in run
            end if
        end if
    end for

    return result
```

Correctness & Complexity:

- Every integer in data belongs to exactly one consecutive run. The inner while loop visits each number at most once across the whole algorithm because we only start a walk at run starts (x with x−1 not in S).

- Building the hash set is Θ(n). The outer loop is Θ(n), and the total work of all inner walks is Θ(n). So overall time is Θ(n).

Example (given): data = [19, 5, 20, 15, 3, 8, 4, 13, 18, 17]
- S = {3,4,5,8,13,15,17,18,19,20}
- Runs found: {3,4,5} ⇒ endpoints 3 and 5; {17,18,19,20} ⇒ endpoints 17 and 20. So result = [3,5,17,20].

7. RadixSort contents after each iteration (LSD, base 10, 4 passes)

Initial array:

9819 4132 2379 5319 7764 9132 7767 4339 6579 1812

Pass 1 — sort by units digit (stable):

4132 9132 1812 7764 7767 9819 2379 5319 4339 6579

Pass 2 — sort by tens digit (stable):

1812 9819 5319 4132 9132 4339 7764 7767 2379 6579

Pass 3 — sort by hundreds digit (stable):

4132 9132 5319 4339 2379 6579 7764 7767 1812 9819

Pass 4 — sort by thousands digit (stable):

1812 2379 4132 4339 5319 6579 7764 7767 9132 9819

Final sorted array after RadixSort: (same as Pass 4)

1812 2379 4132 4339 5319 6579 7764 7767 9132 9819

8. (a) Recommendation when array sorted except for 3 swapped pairs

Recommendation:

Use insertion sort. Insertion sort runs in $\Theta(n + I)$ time where I is the number of inversions in the array. For an almost-sorted array (few swaps), I is small, so insertion sort performs very well in practice and is stable and in-place.

Worst-case complexity for this input:

If three (disjoint) pairs are swapped in an otherwise sorted array, each swap can introduce at most $\Theta(n)$ inversions (a swap of two elements far apart yields Θ(distance) inversions). So three swaps

introduce at most $\Theta(n)$ inversions in total. Therefore $I = \Theta(n)$ in the worst case for three swaps, and insertion sort runs in $\Theta(n + I) = \Theta(n)$.

Therefore insertion sort is the recommended choice and its worst-case time on this specific input (3 swapped pairs) is $\Theta(n)$.

8. (b) When would other algorithms become competitive?

Reasoning:

If s is the number of swapped pairs, in the worst case each swapped pair can create $\Theta(n)$ inversions, so $I = \Theta(s \cdot n)$. Insertion sort costs $\Theta(n + I) = \Theta(n + s \cdot n) = \Theta((s+1) \cdot n)$. Comparison-based sorts like mergesort/heapsort run in $\Theta(n \log n)$ regardless of how many inversions are present.

To make insertion sort no better than an n log n algorithm, we need $s \cdot n = \Theta(n \log n) \Rightarrow s = \Theta(\log n)$.

Conclusion: Once the number of swapped pairs is on the order of $\Theta(\log n)$ (in the worst case), other general-purpose $O(n \log n)$ sorting algorithms become competitive.